

Unity-MCP サンプルプロンプト集

Related: [Define 行列](#) · [API coverage](#) · [MCP README](#)

Cursor 等の MCP クライアントから、Unity MIDI Plugin の MCP ツールを呼び出すときの状況別サンプル文言です。カテゴリ単位の例であり、個別 Tool ID の網羅一覧ではありません。

使い方

- プロジェクトに [Unity-MCP](#) (`com.ivanmurzak.unity.mcp`) を導入し、**Window** → **AI Game Developer** で Cursor 連携を設定する。
- 下記のプロンプトをチャットに貼る（パス・BPM・deviceId などは環境に合わせて書き換える）。
- 任意依存カテゴリは対応する `FEATURE_*` が無いとツール自体が出ない。[define-matrix](#) を参照。
- 送信・モニター・多くの再生操作は **Play Mode 必須**。Edit Mode では `[Error] ... Play Mode ...` が返る。

1. Meta / 診断

いつ使うか: 初めて MCP を触るとき、どの統合が有効か分からないとき、ドキュメントの場所を探るとき、define を安全に足したいとき。

主なツール: `midi-features-status` · `midi-features-enable` · `midi-integration-docs`

Unity MIDI Plugin の MCP 設定を確認して。

- `midi-features-status` で `FEATURE_*` ・ パッケージ ・ asm ・ VST 検出結果を要約する
- Chunity と MPTK の導入手順ドキュメントを `midi-integration-docs` で案内する
define を足す場合は `allowlist` のみ、`confirm=true` で `midi-features-enable` する

VST3 ホストを触りたいが、このリポジトリの MIDI MCP に `vst3-*` は無い想定。
`midi-features-status` で `jp.kshoji.unity.vst3nativehost` の有無を確認し、
`midi-integration-docs topic=vst3` で VST 側計画へ誘導して。

2. Core（初期化・デバイス・送信・モニター）

いつ使うか: 実機 / 仮想 OUT に届くか一発確認したい。Monitor で送受信を切り分けたい。ノート名と番号を変換したい。

主なツール: `midi-init` · `midi-devices-list` · `midi-send-note / cc / pc` · `midi-monitor-read / clear` · `midi-note-utility` · `midi-panic`

Play Mode にしてから:

`midi-init` → `midi-devices-list` → 先頭 OUT に C4 (note=60, velocity=100) を `midi-send-note` / `midi-monitor-read` で NoteOn が見えるか確認して。終わったら `midi-panic`。

CC1 (Modulation) を `channel=0`, `value=64` で送ってモニターに出るか見て。
`deviceId` は `midi-devices-list` の OUT から選んで。

ノート名 D#4 を番号に、番号 72 をノート名に `midi-note-utility` で変換して。

3. SMF / UMP シーケンス・録音

いつ使うか: `.mid` / `.midi2` をアセット化し、`SmfPlayer` で再生したい。テンポマップを見たい。入力を録音して Sequential Asset にしたい。

主なツール: `smf-preview-info` · `smf-import-asset` · `ump-import-asset` · `smf-player-control` · `midi-recorder-control` · `tempo-map-extract` · `ump-sequencer-control` 等

Assets 以下の `sample.mid` を `smf-preview-info` で概要 (トラック数・テンポ・長さ) を出し、問題なければ `smf-import-asset` で `MidiSequenceAsset` を作って。
その後 `smf-player-control` で `SmfPlayer` をシーンに `create / assign` し、
Play Mode で Play → 数秒後 Stop。

いまのシーンに `MidiRecorder` を用意し、Play Mode で `start` → キーボード入力 → `stop` → `save-asset` で録音結果を保存して。

インポート済み Sequence から `tempo-map-extract` でテンポ変更点を一覧して。

4. Gameplay (Router / Filter / Virtual / Settings)

いつ使うか: NoteOn で UnityEvent を発火する雛形をすぐ置きたい。チャンネル／デバイスで絞りたい。仮想デバイスに注入してハンドラ単体を試したい。Project Settings を変えたい。

主なツール: `midi-setup-router` ・ `midi-note-tracker-state` ・ `midi-filter-configure` ・ `midi-virtual-device` ・ `midi-virtual-device-inject` ・ `midi-project-settings-get` / `set`

`MidiInputMap` + `MidiInputRouter` を `midi-setup-router` で作り、
`NoteOn` → `UnityEvent` のバインドを1本追加して。動作確認用に仮想デバイス注入の手順も短く案内し

Play Mode で `midi-virtual-device` を登録し、`NoteOn C4` を注入。
`midi-note-tracker-state` でホールド中ノートが増えるか見て。

`midi-project-settings-get` で現在の MIDI Project Settings を要約し、
変更が必要なら差分を提案してから `midi-project-settings-set` (`confirm` が要る場合は従う)。

5. Timeline / Animator / Input System / Visual Scripting

いつ使うか: Director 上で SMF 再生・録音トラックを組みたい。MIDI で Animator パラメータを動かしたい。Input System の Synthetic Device と往復したい。VS ノード登録だけ済ませたい（グラフ自動編集は対象外）。

主なツール: `midi-timeline-*` ・ `midi-animator-*` ・ `midi-inputsystem-*` ・ `midi-vs-register-nodes` / `midi-vs-status`

前提 **define**: Timeline / Input System / VS は各 `FEATURE_*`

`FEATURE_USE_TIMELINE` 前提で、`midi-timeline-setup-playback` により
`PlayableDirector` + `SmfPlayer` + `Playback Track/Clip` 一式を現在シーンに組んで。
`SequenceAsset` は既存のものを `bind-clip` で割り当てて。

Animator に `midi-animator-add-driver` で `Driver` + `Mapping` を付け、
`CC1` → `Float` パラメータのバインドを `midi-animator-add-binding` で追加。
Play Mode で `midi-animator-read-params` の読み方を教えて。

FEATURE_INPUT_SYSTEM で midi-inputsystem-setup-bridge を実行し、
midi-inputsystem-list-controls と suggest-bindings で .inputactions 向けパスを提案して。

midi-vs-status で登録状況を見て、足りなければ midi-vs-register-nodes。
グラフ編集は手作業でよいので、登録と欠落依存だけ報告して。

6. Scriptable Audio

いつ使うか: Unity 6 Scriptable Audio 上でメトロノームや DSP Sequence をブートストラップし、再生・クロックを検証したい。

主なツール: `midi-sa-bootstrap-metronome` / `smf` / `ump` · `midi-sa-transport` · `midi-sa-clock-mode` · `midi-sa-validate`

前提: `FEATURE_SCRIPTABLE_AUDIO` (Unity 6000.3+)

FEATURE_SCRIPTABLE_AUDIO で metronome を bootstrap (bpm=120) → midi-sa-validate で直し
Play Mode でクリックが聞こえるまで midi-sa-transport を使って。

既存の MidiSequenceAsset を midi-sa-bootstrap-smf で DSP 経路に乗せ、
clock-mode と transport (Play/Stop/Seek) の使い方を短く実演して。

7. Chunity (Chuck)

いつ使うか: 依存や Upstream PR が足りず音が出ない。Bridge + PatchHost を雛形配置したい。ポリフォニ多重発火を避けたい。Chuck Event → MIDI OUT を Monitor で確認したい。Generator / Timeline 連携を触りたい。

主なツール: `midi-chunity-diagnostics` / `validate-scene` / `setup-bridge` / `run-patch` / `set-global` · `poly-setup` · `event-to-midi` · `generator-*` 等

前提: `FEATURE_CHUNITY` (Generator は `FEATURE_CHUNITY_SCRIPTABLE_AUDIO`)

Resource 例: `midi://chunity/scene-status`

音が出ない原因切り分け:

midi-chunity-diagnostics と midi-chunity-validate-scene (Resource midi://chunity/scen Main/Sub/Bridge/PatchHost/define/Upstream PR/WebGL ArraySyncer 注意を報告して。
足りない PR があれば apply-upstream-prs (confirm=true) の要否を判断して。

Bridge + PatchHost を setup-bridge で置き、noteOn を受け取る短い .ck を run-patch。
Play Mode で C4 を送り、グローバル midiNote / noteOn の反応を poll-global で確認して。

ポリフォで二重発火しないよう midi-chunity-poly-setup (useDefaultConvention=false) し、
重なった NoteOn で voiceId / activeNotes が妥当か見て。

Event→MIDI バインドを midi-chunity-event-to-midi で追加し、
パッチから Event 発火 → midi-monitor-read で OUT を確認して。

8. MPTK (Maestro)

いつ使うか: mptk:internal で発音確認したい。SMF Preview Audio (MPTK) 相当を試したい。Pro の Writer / Pipeline / External を触りたい。SA DSP とミラーしたい。

主なツール: midi-mptk-bootstrap / validate-setup / send-test-note / panic · preview-smf ·
global-settings · distance-audio · Pro effects-configure / voice-lifecycle / chord-
progression · writer-* / pipeline-* · dsp-*-mirror · experimental velocity-attenuation

前提: FEATURE_USE_MPTK (Pro は MPTK_PRO)

Resource 例: midi://mptk/setup-report

FEATURE_USE_MPTK で:

Play Mode → midi-init → midi-mptk-bootstrap → validate-setup →
midi-mptk-send-test-note (C4) で発音確認 → midi-mptk-panic。
失敗時は Resource midi://mptk/setup-report の内容を要約して。

MidiSequenceAsset (または .mid) を midi-mptk-preview-smf で Preview Audio (MPTK) 相当まで終わったら panic。

MPTK_PRO 前提: writer-load → writer-play (または write-temp) 。
必要なら external-play で file:// や https:// を再生し、InputAdapter 経由で Monitor も見

Pro Pipeline で ch9 Drop + 簡単な inject ルールを pipeline-configure / edit-rule で組み
再生後に midi://mptk/pipeline/stats を読んで。InputAdapter 併用時は post-rewrite 経路を排

FEATURE_USE_MPTK: midi-mptk-global-settings で runInBackground を get/set。
midi-mptk-distance-audio で DistanceAttenuation (Spatializer Track/Channel とは別) 。
MPTK_PRO: midi-mptk-effects-configure で EnableFilter、
midi-mptk-voice-lifecycle で pause→resume、
midi-mptk-chord-progression で list genre=Pop → play uplifting_pop (生成。midi-chord-s

9. Networking

いつ使うか: UDP Hub/Client の雛形と RTT を見たい。Mirror / NGO / WSN2 ブリッジの足場だけ欲しい (本体パッケージは別途) 。

主なツール: midi-net-hub-client-setup · midi-net-rtt · midi-net-discovery-status · midi-net-bridge-setup · midi-wsnet2-sync-defines

前提: FEATURE_MIDI_NETWORK

Resource 例: midi://network/status

FEATURE_MIDI_NETWORK で loopback Hub/Client を midi-net-hub-client-setup (127.0.0.1) ,
Play Mode + midi-init のあと midi-net-rtt ping=true で RTT を取る。
任意で discovery-status と Resource midi://network/status も。

Hub セットアップ後、framework=mirror (または netcode / wsnet2) で midi-net-bridge-setup。
WSNet2 なら先に midi-wsnet2-sync-defines。ベンダーパッケージはコミットしない前提で足りない

10. MIDI 2.0 / MPE / MIDI-CI

いつ使うか: UMP デバイス列挙と送信、MPE ゾーン確認、Capability Inquiry (Discovery) を試したい。

主なツール: `midi2-devices-list` ・ `midi2-send-ump` / 構造化 `midi2-send-*` ・ `mpe-zone-status` ・ `midi-ci-discover`

Play Mode で `midi2-devices-list (initialize=true)` → 有効な OUT に MIDI 2.0 Note On 相当の UMP (または `midi2-send-channel-voice`) を送り、結果を報告して。

CI 対応デバイスが IN+OUT に繋がっている前提で `midi-ci-discover` を実行し、`discoveredCount` / `MUID` / `capability` フラグを要約して。

MPE ゾーンをセットアップ済み (または `setupIfMissing`) として `mpe-zone-status` の内容を解説し

11. Transport / Foundation (補助)

いつ使うか: RTP / BLE / Nearby / UDP2 セッションの状態確認、出力ルーティングや Clock、レイテンシ校正の入口が欲しいとき。深いハードウェア UI の完全再現は対象外。

主なツール: `midi-rtp-session` ・ `midi-ble-control` ・ `midi-nearby-control` ・ `midi2-udp-session` ・ `midi-transport-status` ・ `midi-output-routing` ・ `midi-clock-sync` / `output` ・ `midi-latency-calibration` ・ `midi-device-selection` 等

`midi-transport-status` で利用可能なトランスポートと注意点を一覧して。
RTP を試すなら `midi-rtp-session` の最小手順だけ示して (深い Companion 設定は対象外) 。

`midi-output-routing` の現状を見て、特定 `deviceId` へ寄せる手順を短く。
必要なら `midi-clock-sync` / `midi-clock-output` の使い分けも一文で。

チートシート（状況 → カテゴリ）

状況	見る節
MCP 自体が動くか / どの FEATURE が効いているか	\$1 Meta
実機に C4 が届くか	\$2 Core
SMF を置いて再生・録音	\$3 SMF
ゲーム側の入力配線	\$4 Gameplay
Timeline / Animator / Input System / VS	\$5
Scriptable Audio でクリックが出るか	\$6
Chuck 統合の診断～発音	\$7
Maestro で発音・Preview・Pro	\$8
ネットワーク同期の足場	\$9
UMP / MPE / CI	\$10
RTP・BLE・Clock などの入口	\$11

VST ホスト操作（`vst3-*`）は本ドキュメントの対象外です。[Unity-VST3-Bridge](#) 側のドキュメントを参照してください。